

Doc. No.:	P0200R0
Date:	2016-01-22
Audience:	Library Evolution Working Group
Reply to:	Yegor Derevenets

A Proposal to Add Y Combinator to the Standard Library

This document proposes to add Y combinator to the C++ standard library. [Y combinator](#) is a well-known high-order function used to implement recursion. Y combinator, accompanied by [C++14 generic lambdas](#), provides a convenient way to define recursive lambda functions.

Motivation

C++11/14 lambdas do not encourage recursion: there is no way to reference the lambda object from the body of the lambda function.

A common workaround for this problem is to use `std::function`, as suggested, e.g., in [these discussions](#):

```
#include <functional>
#include <iostream>

int main() {
    std::function<int(int, int)> gcd = [&](int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    };
    std::cout << gcd(20, 30) << std::endl;
}
```

The solution has several disadvantages:

- Verbosity: we have to write the signature of the lambda function twice: in `std::function` and in the lambda itself.
- Unnecessary overhead: we resort to type erasure, which leads to the likely use of the heap and indirect function calls.
- The solution does not work with generic lambdas.
- The resulting `std::function` object has broken copy semantics: its copies refer to the original object through the lambda capture and become largely useless once the original object is destroyed.

We propose adding to the standard library a `std::y_combinator` function that enables the following fast and clean code, free from the above problems:

```
#include <functional>
#include <iostream>

int main() {
    auto gcd = std::y_combinator([](auto gcd, int a, int b) -> int {
        return b == 0 ? a : gcd(b, a % b);
    });
    std::cout << gcd(20, 30) << std::endl;
}
```

Impact on the Standard

The proposal is a pure library extension. It does not require changes to the standard components. The extension can be implemented in C++11 and C++14.

Design Decisions

The `std::y_combinator` function is a helper function accepting the input callable object of type `Fun` and returning a callable object of type `std::y_combinator_result<Fun>` containing a copy of the input callable. When called, the `std::y_combinator_result<Fun>` object forwards all the arguments to the copy of the input callable, prepending them by a single argument — a wrapped reference to itself.

To construct a `std::y_combinator_result` instance storing a reference to the input callable object, one can combine `std::y_combinator` with `std::ref`:

```
#include <functional>
```

```
#include <iostream>

int main() {
    auto almost_gcd = [](auto gcd, int a, int b) -> int {
        return b == 0 ? a : gcd(b, a % b);
    };
    auto gcd = std::y_combinator(std::ref(almost_gcd));
    std::cout << gcd(20, 30) << std::endl;
}
```

Technical Specification

Changes are relative to [n4567](#). An example implementation of the specification is provided in the [appendix](#).

20.9 Function objects [function.objects]

² Header <functional> synopsis

```
namespace std {
    ...

    // 20.9.X, Y combinator
    template<class Fun> class y_combinator_result;
    template<class Fun> y_combinator_result<decay_t<Fun>> y_combinator(Fun &&fun);
}
```

20.9.X Class template y_combinator_result [yc]

```
template<class Fun>
class y_combinator_result {
public:
    template<class T>
    explicit y_combinator_result(T &&fun);

    template<class ...Args>
    decltype(auto)
    operator()(Args &&...args);
};
```

¹ y_combinator_result template class is a forwarding call wrapper around a target callable object of type `Fun`.

20.9.X.1 y_combinator_result construct [yc.const]

```
template<class T>
explicit y_combinator_result(T &&fun);
```

¹ Effects: Constructs the target callable object of `*this` from `std::forward<T>(fun)`.

20.9.X.2 y_combinator_result invocation [yc.invoke]

```
template<class ...Args>
decltype(auto)
operator()(Args &&...args);
```

¹ Returns: `f(std::ref(*this), std::forward<Args>(args)...)...`, where `f` is the target callable object of `*this`.

20.9.X.3 y_combinator_result helper function [yc.helper]

```
template<class Fun> y_combinator_result<decay_t<Fun>> y_combinator(Fun &&fun);
```

¹ Returns: `y_combinator_result<decay_t<Fun>>(std::forward<Fun>(fun))`.

Discussion

- Unlike `reference_wrapper` and `function`, `y_combinator_result` deliberately does not define `result_type`, `argument_type`, `first_argument_type`, `second_argument_type` typedefs. It is generally impossible to infer the corresponding types for an arbitrary target function object type. One can do this reliably only for function types and function pointer types. However, functions already have a name that they can use to call themselves. So, it is unclear why one would like to use Y combinator with them.
- `y_combinator_result` deliberately does not support member function pointers. The reason, as in the previous item, is the lack

of sensible use cases: member functions have a name and can call themselves.

- As [noted](#) by Richard Smith, an alternative way to support recursive lambdas is to extend the core language, e.g.:

```
auto gcd = [] self (int a, int b) -> int {  
    return b == 0 ? a : self(b, a % b)  
};
```

, where `self` is the identifier used to refer to the lambda object within the lambda.

Appendix: Example Implementation

An example implementation of `std::y_combinator_result` and `std::y_combinator` follows.

```
#include <functional>  
#include <utility>  
  
namespace std {  
  
template<class Fun>  
class y_combinator_result {  
    Fun fun_;  
public:  
    template<class T>  
    explicit y_combinator_result(T &&fun): fun_(std::forward<T>(fun)) {}  
  
    template<class ...Args>  
    decltype(auto) operator()(Args &&...args) {  
        return fun_(std::ref(*this), std::forward<Args>(args)...);  
    }  
};  
  
template<class Fun>  
decltype(auto) y_combinator(Fun &&fun) {  
    return y_combinator_result<std::decay_t<Fun>>(std::forward<Fun>(fun));  
}  
  
} // namespace std
```